Reflected XSS Protected by Very Strict CSP with Dangling Markup Attack

1. Overview

Field	Details
Vulnerability	Reflected Cross-Site Scripting
Related Technique	Dangling Markup Injection
Security Mechanism	Strict Content Security Policy
Category	HTML Injection
Severity	Context-dependent
Lab Platform	PortSwigger Web Security Academy

2. Description

Dangling markup injection is a technique that abuses incomplete or improperly terminated HTML markup to influence how a browser parses subsequent content.

Unlike conventional XSS, this technique does not necessarily require JavaScript execution.

An attacker may exploit an injection point to introduce HTML markup that changes the interpretation of the remaining document.

In certain situations, dangling markup can expose information contained in subsequent HTML content, even when a strict Content Security Policy prevents conventional script execution.

This lab demonstrates how HTML parsing weaknesses can create security risks beyond traditional JavaScript-based XSS.

3. Root Cause

The underlying issue is unsafe reflection of user-controlled input into HTML markup.

Potential contributing factors include:

- Inadequate HTML output encoding.
- Improper handling of HTML attribute boundaries.
- Incomplete sanitization of user-controlled input.
- Reliance on CSP as the primary XSS defense.
- Sensitive information placed in HTML where injected markup may affect parsing.

4. Proof of Concept (PoC)

Authorized lab testing methodology:

1. Open the assigned lab.
2. Identify the reflected input parameter.
3. Inspect the HTTP response and determine the exact HTML insertion context.
4. Review the CSP response header.
5. Identify whether normal script execution is restricted.
6. Examine whether user input can interfere with subsequent HTML parsing.
7. Investigate the possibility of dangling markup and its effect on document interpretation.
8. Use harmless test content to verify the parsing behavior.
9. Document any information exposure using only authorized lab data.

Example HTML parsing context:

```
None
<div>
    USER_INPUT
</div>
```

This illustrates a basic reflection point. It is not a working dangling markup payload.

5. Security Impact

Depending on the vulnerable context, dangling markup injection may result in:

- Unauthorized exposure of information contained in subsequent HTML.
- Manipulation of document parsing.
- Unexpected browser resource requests.
- Exposure of sensitive values embedded in page content.
- Potential bypass of protections that only address JavaScript execution.

6. Recommended Remediation

- Apply context-aware HTML output encoding.
- Avoid inserting untrusted input directly into HTML markup.
- Use safe DOM APIs and trusted templating systems.
- Sanitize HTML with a maintained security library when rich HTML input is required.
- Keep sensitive information out of unnecessary HTML contexts.
- Use strict CSP as defense in depth.
- Apply appropriate referrer and resource-loading policies.
- Test malformed markup and HTML parsing edge cases.

7. Verification

1. Repeat the original injection test.
2. Confirm that untrusted input cannot alter surrounding HTML markup.
3. Verify that sensitive page content cannot be exposed through malformed markup.
4. Test the CSP policy independently.
5. Inspect unexpected browser resource requests.
6. Confirm that legitimate content renders correctly.

8. References

- PortSwigger: Dangling Markup Injection
- PortSwigger: Content Security Policy
- OWASP XSS Prevention Cheat Sheet
- MDN: Content Security Policy

9. Disclaimer

For educational purposes and authorized security testing only.